ASSIGNMENT 3 – C# FUNDAMENTALS

Module 1 – Complete Revision Handbook

Technology: C# / .NET
Purpose: Training Revision, Practical Learning and Interview Preparation

---

CONTENTS

1. Introduction to C#
2. C# Program Structure
3. Variables and Constants
4. Data Types
5. Type Conversion and Type Casting
6. Operators
7. Conditional Statements
8. Looping Statements
9. Arrays
10. Strings
11. Methods
12. Object-Oriented Programming
13. Classes and Objects
14. Constructors
15. Inheritance
16. Polymorphism, Abstraction and Encapsulation
17. Interfaces
18. Access Modifiers
19. Exception Handling
20. Collections and Generics

---

1. INTRODUCTION TO C#

1.1 What is C#?

C# (pronounced C-Sharp) is a modern, object-oriented, general-purpose programming language developed by Microsoft.

It is used to develop:

- Web applications.
- Desktop applications.
- REST APIs.
- Enterprise applications.
- Cloud applications.
- Games.

C# runs on the .NET platform.

1.2 Features of C#

1. Object-Oriented: Supports classes, objects, inheritance and polymorphism.
2. Strongly Typed: Every variable has a defined data type.
3. Platform Independent: Modern .NET applications support Windows, Linux and macOS.
4. Automatic Memory Management: Garbage Collector manages memory.
5. Exception Handling: Supports try, catch and finally.
6. Asynchronous Programming: Supports async and await.
7. Rich Libraries: Provides built-in classes and methods.

1.3 C# vs .NET

C#	.NET
Programming language	Development platform
Used to write code	Used to build and execute applications
Defines syntax and rules	Provides runtime and libraries
Example: Classes, methods	Example: CLR, BCL

1.4 Important .NET Components

Component	Description
CLR	Common Language Runtime
BCL	Base Class Library
JIT	Just-In-Time Compiler
CTS	Common Type System

CLS	Common Language Specification
SDK	Software Development Kit
GC	Garbage Collection

CLR

CLR manages execution of .NET applications.

Responsibilities:

- Memory management.
- Garbage collection.
- Exception handling.
- Type safety.
- Thread management.

JIT

JIT converts Intermediate Language (IL) into native machine code during execution.

Compilation Process

C# Source Code → C# Compiler → Intermediate Language → CLR/JIT → Machine Code → Execution.

Garbage Collection

Garbage Collection automatically identifies unreachable objects and reclaims their memory.

---

2. C# PROGRAM STRUCTURE

2.1 Basic Program Structure

```
None
using System;

namespace MyApplication
{
    class Program
    {
```

```
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World");
        }
    }
}
```

Output:

```
None
Hello World
```

2.2 Components

Component	Description
using System	Imports System namespace
namespace	Organizes related types
class Program	Declares a class
static	Allows calling without an object
void	No return value
Main()	Traditional entry point
Console.WriteLine()	Displays output

2.3 Namespace

A namespace organizes related classes, interfaces and other types.

```
None
namespace StudentManagement
{
    class Student
    {
        public void Display()
        {
            Console.WriteLine("Student Details");
```

```
        }
    }
}
```

2.4 Class

A class is a blueprint for creating objects.

```
None
class Student
{
    public string Name = "Divya";

    public void Display()
    {
        Console.WriteLine(Name);
    }
}
```

2.5 Main Method

The traditional entry point of a C# executable application.

```
None
static void Main()
{
    Console.WriteLine("Application Started");
}
```

Common signatures:

```
None
static void Main()
static void Main(string[] args)
static int Main()
static async Task Main()
```

2.6 Statements

A statement is an instruction executed by the program.

```
None
int age = 25;
Console.WriteLine(age);
```

2.7 Comments

Single-line:

```
None
// This is a comment
```

Multi-line:

```
None
/*
This is a
multi-line comment
*/
```

XML documentation:

```
None
/// <summary>
/// Displays student details.
/// </summary>
public void Display()
{
    Console.WriteLine("Student");
}
```

2.8 Console Input and Output

Method	Purpose
Console.Write()	Displays without a new line

Console.WriteLine()	Displays with a new line
Console.ReadLine()	Reads a line of input
Console.ReadKey()	Reads a key

Example:

```
None
Console.Write("Enter name: ");
string name = Console.ReadLine() ?? "";

Console.WriteLine($"Welcome {name}");
```

2.9 Top-Level Statements

Modern C# supports writing executable statements without explicitly declaring Program and Main.

```
None
Console.WriteLine("Hello from Modern C#");
```

The compiler generates the entry-point structure.

---

3. VARIABLES AND CONSTANTS

3.1 What is a Variable?

A variable is a named memory location used to store data that can change during program execution.

Syntax:

```
None
dataType variableName = value;
```

Example:

```
None
int age = 25;
string name = "Divya";

age = 26;

Console.WriteLine(age);
```

Output:

```
None
26
```

3.2 Variable Declaration and Initialization

Declaration: Creating a variable.

```
None
int age;
```

Initialization: Assigning an initial value.

```
None
age = 25;
```

Both together:

```
None
int age = 25;
```

3.3 Types of Variables Based on Declaration

Type	Description
Local variable	Declared inside a method or block
Instance variable	Non-static field belonging to an object
Static variable	Shared by all instances of a class

Example:

```
None
class Employee
{
    public string name = "Divya";  // Instance field

    public static string company = "HCL"; // Static field

    public void Display()
    {
        int age = 25; // Local variable
        Console.WriteLine(age);
    }
}
```

3.4 Constants

A constant is a value that cannot be changed after declaration.

```
None
const double PI = 3.14159;
```

Example:

```
None
class Program
{
    static void Main()
    {
        const int MAX = 100;

        Console.WriteLine(MAX);
    }
}
```

3.5 Readonly

A readonly field can be assigned during declaration or inside a constructor. It cannot be reassigned after the object has been constructed.

```
None
class Employee
{
    public readonly int employeeId;

    public Employee(int id)
    {
        employeeId = id;
    }
}
```

3.6 var Keyword

The `var` keyword allows the compiler to infer the variable's type from its initialization expression.

```
None
var name = "Divya";
var age = 25;
var salary = 35000.50;
```

The inferred types are string, int and double.

Important: `var` does not mean dynamically typed. The type is determined at compile time.

3.7 Naming Rules

- Names can contain letters, digits and underscores.
- Names cannot start with a digit.
- C# keywords cannot be used directly as identifiers.
- C# is case-sensitive.
- Use meaningful names.

Examples:

```
None
int studentAge = 21;
string employeeName = "Divya";
```

---

4. DATA TYPES

A data type defines the type of value a variable can store.

C# data types are broadly classified into value types and reference types.

4.1 Value Types

Value types store their values directly.

Examples:

Type	Description	Example
int	32-bit integer	100
long	64-bit integer	100000L
short	16-bit integer	100
byte	8-bit unsigned integer	255
float	32-bit floating point	10.5f
double	64-bit floating point	10.5
decimal	High-precision decimal	99.99m
char	Single character	'A'
bool	Boolean	true

Example:

```
None
int age = 25;
double salary = 35000.50;
char grade = 'A';
bool isActive = true;

Console.WriteLine(age);
Console.WriteLine(salary);
Console.WriteLine(grade);
Console.WriteLine(isActive);
```

4.2 Reference Types

Reference types store references to objects.

Examples:

- string
- object
- class
- array
- interface
- delegate

Example:

```
None
string name = "Divya";

int[] numbers = { 10, 20, 30 };
```

4.3 Value Types vs Reference Types

Value Types	Reference Types
Store values directly	Store references to objects
Examples: int, bool, struct	Examples: class, string, array
Assignment copies the value	Assignment copies the reference
Cannot normally contain null	Can contain null

4.4 String

String stores a sequence of characters.

```
None
string name = "Divya Xavier";
```

Strings are immutable, meaning their contents cannot be modified after creation. Operations that appear to modify a string produce another string.

4.5 Object

`object` is the ultimate base type of all C# types.

```
None
object value = 100;
value = "Hello";
```

The variable can refer to objects of different types, although each assignment still has a specific runtime type.

4.6 Nullable Value Types

Nullable value types can hold a normal value or null.

```
None
int? age = null;

age = 25;
```

The `?` allows a value type to represent null.

---

5. TYPE CONVERSION AND TYPE CASTING

Type conversion means changing a value from one data type to another.

5.1 Implicit Conversion

Automatic conversion performed when the destination type can safely represent the source value.

```
None
int number = 100;
double result = number;

Console.WriteLine(result);
```

Output:

```
None
100
```

5.2 Explicit Conversion

Manual conversion using casting.

```
None
double salary = 35000.75;

int amount = (int)salary;

Console.WriteLine(amount);
```

Output:

```
None
35000
```

The fractional part is discarded.

5.3 Convert Methods

The Convert class provides methods to convert values.

```
None
string age = "25";

int result = Convert.ToInt32(age);

Console.WriteLine(result);
```

5.4 Parse Method

Parse converts a valid string into a specified data type.

```
None
string number = "100";
```

```
int result = int.Parse(number);
```

Invalid input can cause a FormatException.

5.5 TryParse Method

TryParse attempts conversion without throwing an exception for ordinary invalid input.

```
None
string input = "100";

if (int.TryParse(input, out int result))
{
    Console.WriteLine(result);
}
else
{
    Console.WriteLine("Invalid input");
}
```

5.6 Differences

Parse	TryParse	Convert
Converts strings	Attempts conversion	Converts between supported types
Throws exception on invalid format	Returns false on invalid format	May throw conversion exceptions
Returns converted value	Uses out parameter	Returns converted value
Useful for known-valid input	Useful for user input	Useful for general conversions

---

6. OPERATORS

Operators perform operations on variables and values.

6.1 Arithmetic Operators

Operator	Purpose	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus	a % b

Example:

```
None
int a = 10;
int b = 3;

Console.WriteLine(a + b);
Console.WriteLine(a % b);
```

Output:

```
None
13
1
```

6.2 Relational Operators

Used to compare values.

Operator	Meaning
==	Equal
!=	Not equal
>	Greater than
<	Less than
>=	Greater than or equal

<=	Less than or equal

Example:

```
None
int a = 10;
int b = 20;

Console.WriteLine(a < b);
```

Output:

```
None
True
```

6.3 Logical Operators

Operator	Meaning
&&	Logical AND
||	Logical OR
!	Logical NOT

Example:

```
None
int age = 25;

if (age >= 18 && age <= 60)
{
    Console.WriteLine("Eligible");
}
```

6.4 Assignment Operators

Operator	Example	Equivalent
=	a = 10	Assign
+=	a += 5	a = a + 5

-=	a -= 5	a = a - 5
*=	a *= 2	a = a * 2
/=	a /= 2	a = a / 2
%=	a %= 2	a = a % 2

6.5 Increment and Decrement

```
None
int a = 5;

a++;
a--;

Console.WriteLine(a);
```

Output:

```
None
5
```

- `a++` increments the value.
- `a--` decrements the value.
- `++a` is pre-increment.
- `a++` is post-increment.

6.6 Ternary Operator

A shorthand alternative to a simple if-else expression.

Syntax:

```
None
condition ? valueIfTrue : valueIfFalse;
```

Example:

```
None
int age = 20;

string result = age >= 18 ? "Adult" : "Minor";

Console.WriteLine(result);
```

Output:

```
None
Adult
```

6.7 Null-Coalescing Operator

Returns the left value if it is not null; otherwise returns the right value.

```
None
string? name = null;

string result = name ?? "Guest";

Console.WriteLine(result);
```

Output:

```
None
Guest
```

---

7. CONDITIONAL STATEMENTS

Conditional statements execute code based on conditions.

7.1 if Statement

Executes code when a condition is true.

```
None
int age = 20;

if (age >= 18)
{
    Console.WriteLine("Eligible to vote");
}
```

7.2 if-else Statement

```
None
int number = 10;

if (number % 2 == 0)
{
    Console.WriteLine("Even");
}
else
{
    Console.WriteLine("Odd");
}
```

Output:

```
None
Even
```

7.3 else-if Ladder

Used to check multiple conditions.

```
None
int marks = 85;

if (marks >= 90)
{
    Console.WriteLine("A Grade");
}
```

```
else if (marks >= 75)
{
    Console.WriteLine("B Grade");
}
else if (marks >= 50)
{
    Console.WriteLine("C Grade");
}
else
{
    Console.WriteLine("Fail");
}
```

Output:

```
None
B Grade
```

7.4 Nested if

An if statement inside another if statement.

```
None
int age = 25;
bool hasLicense = true;

if (age >= 18)
{
    if (hasLicense)
    {
        Console.WriteLine("Can drive");
    }
}
```

7.5 switch Statement

Used when comparing a value against multiple possible cases.

```
None
int day = 2;

switch (day)
{
    case 1:
        Console.WriteLine("Monday");
        break;

    case 2:
        Console.WriteLine("Tuesday");
        break;

    default:
        Console.WriteLine("Invalid day");
        break;
}
```

Output:

```
None
Tuesday
```

7.6 Difference Between if-else and switch

if-else	switch
Supports complex conditions	Matches a value against cases
Supports relational operators	Uses case patterns and matching
Useful for ranges	Useful for multiple alternatives
Can evaluate different expressions	Uses a switch expression

---

8. LOOPING STATEMENTS

Loops execute a block of code repeatedly.

8.1 for Loop

Used when the number of iterations is generally known.

```
None
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine(i);
}
```

Output:

```
None
1
2
3
4
5
```

8.2 while Loop

Executes while the condition is true.

```
None
int i = 1;

while (i <= 5)
{
    Console.WriteLine(i);
    i++;
}
```

8.3 do-while Loop

Executes the block at least once before checking the condition.

```
None
int i = 1;
```

```
do
{
    Console.WriteLine(i);
    i++;
}
while (i <= 5);
```

8.4 foreach Loop

Used to iterate through collections and arrays.

```
None
int[] numbers = { 10, 20, 30 };

foreach (int number in numbers)
{
    Console.WriteLine(number);
}
```

8.5 break Statement

Terminates the loop immediately.

```
None
for (int i = 1; i <= 5; i++)
{
    if (i == 3)
        break;

    Console.WriteLine(i);
}
```

Output:

```
None
1
2
```

8.6 continue Statement

Skips the current iteration and proceeds to the next iteration.

```
None
for (int i = 1; i <= 5; i++)
{
    if (i == 3)
        continue;

    Console.WriteLine(i);
}
```

Output:

```
None
1
2
4
5
```

8.7 Loop Comparison

Loop	Description
for	Usually used when iterations are known
while	Checks condition before execution
do-while	Executes at least once
foreach	Iterates through a collection

---

9. ARRAYS

9.1 What is an Array?

An array is a collection of elements of the same data type stored under one variable name.

Array indexes start from zero.

9.2 Declaration

```
None
int[] numbers;
```

9.3 Initialization

```
None
int[] numbers = { 10, 20, 30, 40 };
```

9.4 Accessing Elements

```
None
Console.WriteLine(numbers[0]);
```

Output:

```
None
10
```

9.5 Updating Elements

```
None
numbers[0] = 100;
```

9.6 Array Length

```
None
Console.WriteLine(numbers.Length);
```

Output:

```
None
4
```

9.7 Iterating Through an Array

```
None
int[] numbers = { 10, 20, 30 };

for (int i = 0; i < numbers.Length; i++)
{
    Console.WriteLine(numbers[i]);
}
```

9.8 Multidimensional Array

A multidimensional array stores data in multiple dimensions.

Example: Two-dimensional array.

```
None
int[,] matrix =
{
    { 1, 2 },
    { 3, 4 }
};

Console.WriteLine(matrix[0, 1]);
```

Output:

```
None
2
```

9.9 Jagged Array

A jagged array is an array of arrays. Each inner array can have a different length.

```
None
int[][] numbers =
{
    new int[] { 1, 2 },
    new int[] { 3, 4, 5 }
};
```

```
Console.WriteLine(numbers[1][2]);
```

Output:

```
None
5
```

9.10 Useful Array Methods

```
None
int[] numbers = { 30, 10, 20 };

Array.Sort(numbers);
Array.Reverse(numbers);

Console.WriteLine(Array.IndexOf(numbers, 20));
```

Important methods:

- Array.Sort()
- Array.Reverse()
- Array.IndexOf()
- Array.Copy()
- Array.Clear()

---

10. STRINGS

10.1 What is a String?

A string is a sequence of characters represented by the `string` type.

```
None
string name = "Divya";
```

Strings are reference types and immutable.

10.2 String Concatenation

Combining multiple strings using the + operator.

```
None
string first = "Hello";
string second = "World";

string result = first + " " + second;

Console.WriteLine(result);
```

Output:

```
None
Hello World
```

10.3 String Interpolation

Using $ to insert variable values into strings.

```
None
string name = "Divya";
int age = 25;

Console.WriteLine($"My name is {name} and my age is {age}");
```

10.4 Important String Methods

Method	Purpose
Length	Gets string length
ToUpper()	Converts to uppercase
ToLower()	Converts to lowercase
Trim()	Removes leading and trailing whitespace
Contains()	Checks whether text exists

StartsWith()	Checks starting text
EndsWith()	Checks ending text
Replace()	Replaces text
Substring()	Extracts part of a string
Split()	Splits a string
IndexOf()	Finds the index of a value
Equals()	Compares strings

Example:

```
None
string name = " Divya Xavier ";

Console.WriteLine(name.Trim());
Console.WriteLine(name.ToUpper());
Console.WriteLine(name.Contains("Xavier"));
```

10.5 String Immutability

Strings cannot be modified after creation.

```
None
string name = "Hello";

name = name + " World";
```

A new string is created and assigned to the variable.

10.6 StringBuilder

StringBuilder is useful when repeatedly modifying text.

```
None
using System.Text;

StringBuilder sb = new StringBuilder();
```

```
sb.Append("Hello");
sb.Append(" World");

Console.WriteLine(sb.ToString());
```

Output:

```
None
Hello World
```

String vs StringBuilder

String	StringBuilder
Immutable	Mutable
Creates new strings during modification	Modifies its internal character buffer
Suitable for simple text operations	Suitable for repeated modifications
System.String	System.Text.StringBuilder

---

11. METHODS

11.1 What is a Method?

A method is a block of code that performs a specific task.

Methods help improve code reusability, readability and maintainability.

11.2 Method Syntax

```
None
returnType MethodName(parameters)
{
    // Method body
}
```

11.3 Method Without Parameters

```
None
void Display()
{
    Console.WriteLine("Welcome");
}
```

11.4 Method With Parameters

```
None
void Greet(string name)
{
    Console.WriteLine($"Hello {name}");
}
```

Calling the method:

```
None
Greet("Divya");
```

Output:

```
None
Hello Divya
```

11.5 Method With Return Value

```
None
int Add(int a, int b)
{
    return a + b;
}
```

Calling:

```
None
int result = Add(10, 20);
```

```
Console.WriteLine(result);
```

Output:

```
None
30
```

11.6 Method Overloading

Method overloading means defining multiple methods with the same name but different parameter lists.

```
None
int Add(int a, int b)
{
    return a + b;
}

int Add(int a, int b, int c)
{
    return a + b + c;
}

double Add(double a, double b)
{
    return a + b;
}
```

The compiler selects the appropriate overload based on the arguments.

Changing only the return type is not sufficient for method overloading.

11.7 Optional Parameters

Optional parameters have default values.

```
None
void Greet(string name = "Guest")
{
    Console.WriteLine($"Hello {name}");
}
```

Calls:

```
None
Greet();
Greet("Divya");
```

11.8 Named Arguments

Named arguments specify parameter names while calling a method.

```
None
void Display(string name, int age)
{
    Console.WriteLine($"{name} - {age}");
}

Display(age: 25, name: "Divya");
```

11.9 ref and out Parameters

ref: The variable must be initialized before passing it.

```
None
void Update(ref int number)
{
    number = 100;
}

int value = 10;
Update(ref value);

Console.WriteLine(value);
```

Output:

```
None
100
```

out: The method must assign a value to the out parameter before returning normally.

```
None
void GetValue(out int number)
{
    number = 100;
}

GetValue(out int value);

Console.WriteLine(value);
```

Output:

```
None
100
```

11.10 Method vs Function

In C#, the term method is commonly used for a function associated with a class, struct or other type. Methods may return a value or return void.

---

12. OBJECT-ORIENTED PROGRAMMING (OOP)

12.1 What is OOP?

Object-Oriented Programming is a programming approach that organizes code around classes and objects.

It helps develop reusable, maintainable and structured applications.

12.2 Four Pillars of OOP

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

12.3 Advantages of OOP

- Code reusability.
- Better organization.
- Easier maintenance.
- Data protection.
- Flexibility and extensibility.

12.4 Real-World Example

Consider a banking application.

- Class: BankAccount
- Object: Customer's bank account
- Data: AccountNumber, Balance
- Methods: Deposit(), Withdraw()

OOP allows the account's data and operations to be organized together.

---

13. CLASSES AND OBJECTS

13.1 What is a Class?

A class is a blueprint used to create objects.

It contains fields, properties, methods and constructors.

13.2 What is an Object?

An object is an instance of a class.

It represents an entity with data and behaviour.

13.3 Example

```
None
class Student
{
    public string Name = "Divya";
    public int Age = 25;

    public void Display()
    {
        Console.WriteLine(Name);
        Console.WriteLine(Age);
    }
}

class Program
{
    static void Main()
    {
        Student student = new Student();

        student.Display();
    }
}
```

Output:

```
None
Divya
25
```

13.4 Fields

Fields are variables declared inside a class or struct.

```
None
class Employee
{
    public int employeeId;
    public string employeeName = "";
}
```

13.5 Properties

Properties provide controlled access to fields or other data.

```
None
class Employee
{
    public string Name { get; set; }

    public int Age { get; set; }
}
```

13.6 Auto-Implemented Properties

The compiler generates the backing storage.

```
None
public string Email { get; set; }
```

13.7 Object Initialization

```
None
Student student = new Student
{
    Name = "Divya",
    Age = 25
};
```

13.8 Static Members

Static members belong to the class rather than individual objects.

```
None
class Employee
{
    public static string Company = "HCL";
}
```

Access:

```
None
Console.WriteLine(Employee.Company);
```

---

14. CONSTRUCTORS

14.1 What is a Constructor?

A constructor is a special class member that executes when an object is created.

It initializes the object.

Characteristics:

- Has the same name as the class.
- Has no return type.
- Can accept parameters.
- Can be overloaded.

14.2 Default Constructor

A parameterless constructor.

```
None
class Student
{
    public Student()
    {
        Console.WriteLine("Student Created");
    }
}
```

14.3 Parameterized Constructor

```
None
class Student
{
    public string Name;
```

```
    public Student(string name)
    {
        Name = name;
    }
}
```

Usage:

```
None
Student student = new Student("Divya");

Console.WriteLine(student.Name);
```

Output:

```
None
Divya
```

14.4 Constructor Overloading

Multiple constructors with different parameter lists.

```
None
class Employee
{
    public string Name;
    public int Age;

    public Employee()
    {
        Name = "Unknown";
        Age = 0;
    }

    public Employee(string name, int age)
    {
        Name = name;
        Age = age;
```

```
    }
}
```

14.5 Static Constructor

A static constructor initializes static data.

```
None
class Employee
{
    public static string Company;

    static Employee()
    {
        Company = "HCL";
    }
}
```

A static constructor:

- Has no access modifier.
- Has no parameters.
- Cannot be called directly.
- Executes automatically before the type is first used.

14.6 Constructor vs Method

Constructor	Method
Initializes objects	Performs operations
Same name as class	Can have any valid method name
Has no return type	Can return a value
Called during object creation	Called explicitly or through other code
Cannot be inherited	Can be inherited

---

15. INHERITANCE

15.1 What is Inheritance?

Inheritance allows one class to acquire accessible members from another class.

It promotes code reuse.

The existing class is called the base class or parent class.

The derived class is called the child class.

15.2 Syntax

None

```
class Parent
{
    // Parent members
}

class Child : Parent
{
    // Child members
}
```

15.3 Example

None

```
class Employee
{
    public void DisplayEmployee()
    {
        Console.WriteLine("Employee Details");
    }
}

class Manager : Employee
{
    public void DisplayManager()
    {
        Console.WriteLine("Manager Details");
    }
}
```

Usage:

```
None
Manager manager = new Manager();

manager.DisplayEmployee();
manager.DisplayManager();
```

Output:

```
None
Employee Details
Manager Details
```

15.4 Types of Inheritance

Type	Description
Single	One child inherits one parent
Multilevel	A class inherits from a derived class
Hierarchical	Multiple child classes inherit one parent
Multiple	One class inherits from multiple base classes

C# supports single class inheritance. Multiple inheritance of implementation through classes is not supported, but multiple interfaces can be implemented.

15.5 base Keyword

The `base` keyword accesses members of the immediate base class.

```
None
class Employee
{
    public Employee(string name)
    {
        Console.WriteLine(name);
    }
}
```

```
class Manager : Employee
{
    public Manager(string name) : base(name)
    {
        Console.WriteLine("Manager Created");
    }
}
```

15.6 Important Points

- Inheritance uses the colon (`:`).
- Constructors are not inherited.
- Private members cannot be directly accessed by derived classes.
- A sealed class cannot be inherited.
- A derived class can override eligible virtual methods.

---

16. POLYMORPHISM, ABSTRACTION AND ENCAPSULATION

16.1 Polymorphism

Polymorphism means one interface or method name can represent different behaviours.

There are two commonly taught forms.

A. Compile-Time Polymorphism

Achieved through method overloading.

```
None
class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
```

```
    }

    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
}
```

B. Runtime Polymorphism

Achieved through method overriding using virtual and override.

```
None
class Animal
{
    public virtual void Sound()
    {
        Console.WriteLine("Animal Sound");
    }
}

class Dog : Animal
{
    public override void Sound()
    {
        Console.WriteLine("Bark");
    }
}
```

Usage:

```
None
Animal animal = new Dog();

animal.Sound();
```

Output:

```
None
Bark
```

16.2 virtual Keyword

The virtual keyword allows a derived class to override a method or property.

16.3 override Keyword

The override keyword provides a new implementation of an eligible virtual or abstract member.

16.4 new Keyword in Method Hiding

The `new` keyword hides an inherited member instead of overriding it.

```
None
class Parent
{
    public void Display()
    {
        Console.WriteLine("Parent");
    }
}

class Child : Parent
{
    public new void Display()
    {
        Console.WriteLine("Child");
    }
}
```

16.5 Abstraction

Abstraction means hiding implementation details and exposing only essential functionality.

It can be achieved using abstract classes and interfaces.

Abstract Class

```
None
abstract class Shape
{
    public abstract void Draw();

    public void Display()
    {
        Console.WriteLine("Shape");
    }
}

class Circle : Shape
{
    public override void Draw()
    {
        Console.WriteLine("Drawing Circle");
    }
}
```

Important:

- An abstract class cannot be instantiated directly.
- It can contain abstract and non-abstract members.
- A derived concrete class must implement inherited abstract members.

16.6 Encapsulation

Encapsulation means bundling data and methods together while controlling access to internal data.

Example:

```
None
class BankAccount
{
    private double balance;

    public void Deposit(double amount)
    {
        if (amount > 0)
        {
```

```
            balance += amount;
        }
    }

    public double GetBalance()
    {
        return balance;
    }
}
```

Usage:

```
None
BankAccount account = new BankAccount();

account.Deposit(5000);

Console.WriteLine(account.GetBalance());
```

Output:

```
None
5000
```

The private balance field cannot be accessed directly outside the class.

16.7 Four Pillars Comparison

Concept	Meaning	Main Purpose
Encapsulation	Hiding internal data	Data protection
Abstraction	Hiding implementation details	Simplifying complexity
Inheritance	Acquiring base class members	Code reuse
Polymorphism	Different behaviours through a common contract	Flexibility

---

17. INTERFACES

17.1 What is an Interface?

An interface defines a contract that implementing types must fulfil.

It specifies members that implementing classes or structs must implement, subject to language features such as default interface implementations.

Interfaces are commonly used to achieve abstraction and loose coupling.

17.2 Syntax

```
None
interface IStudent
{
    void Display();
}
```

17.3 Implementation

```
None
interface IStudent
{
    void Display();
}

class Student : IStudent
{
    public void Display()
    {
        Console.WriteLine("Student Details");
    }
}
```

Usage:

```
None
IStudent student = new Student();
```

```
student.Display();
```

Output:

```
None
Student Details
```

17.4 Multiple Interface Implementation

A class can implement multiple interfaces.

```
None
interface IPrint
{
    void Print();
}

interface IScan
{
    void Scan();
}

class Machine : IPrint, IScan
{
    public void Print()
    {
        Console.WriteLine("Printing");
    }

    public void Scan()
    {
        Console.WriteLine("Scanning");
    }
}
```

17.5 Interface vs Abstract Class

Interface	Abstract Class

Defines a contract	Defines a common base type
Multiple interfaces can be implemented	Only one class base can be inherited
Cannot have instance fields	Can have instance fields
Can contain abstract members and default implementations	Can contain abstract and implemented members
Supports implementation by classes and structs	Class inheritance is required

17.6 Real-World Example

In a MovieTicketBooking application:

- `IBookingRepository` defines database operations.
- `BookingRepository` implements those operations.
- Services depend on the interface instead of a concrete implementation.

This supports loose coupling and easier testing.

---

18. ACCESS MODIFIERS

Access modifiers define where a class or member can be accessed.

18.1 Types of Access Modifiers

Modifier	Accessibility
public	Accessible wherever permitted by the containing type
private	Accessible only within the containing type
protected	Accessible within the containing type and derived types
internal	Accessible within the same assembly
protected internal	Same assembly OR derived types
private protected	Same assembly AND derived types

18.2 Example

```
None
class Employee
{
    public string Name = "Divya";

    private int salary = 30000;

    protected int age = 25;

    internal string department = "IT";
}
```

18.3 Important Differences

- `public`: Widest accessibility.
- `private`: Restricted access.
- `protected`: Supports derived classes.
- `internal`: Restricted to the same assembly.
- `protected internal`: Either same assembly or derived type.
- `private protected`: Derived type within the same assembly.

18.4 Default Accessibility

- Class members are private by default.
- Top-level classes are internal by default.
- Struct members are private by default.

---

19. EXCEPTION HANDLING

19.1 What is an Exception?

An exception is an error condition that occurs during program execution and disrupts the normal flow of execution.

Examples:

- Division by zero.
- Invalid input.
- File not found.

- Null reference access.

19.2 try-catch

The try block contains code that may produce an exception.

The catch block handles the exception.

```
None
try
{
    int a = 10;
    int b = 0;

    int result = a / b;
}
catch (DivideByZeroException ex)
{
    Console.WriteLine(ex.Message);
}
```

19.3 Multiple catch Blocks

```
None
try
{
    int number = int.Parse("ABC");
}
catch (FormatException)
{
    Console.WriteLine("Invalid format");
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

Specific exception handlers should generally appear before broader handlers.

19.4 finally Block

The finally block executes when control leaves the try-catch construct, including when an exception is handled or propagates.

```
None
try
{
    Console.WriteLine("Try block");
}
catch (Exception)
{
    Console.WriteLine("Catch block");
}
finally
{
    Console.WriteLine("Finally block");
}
```

Output:

```
None
Try block
Finally block
```

19.5 throw Keyword

The throw keyword is used to raise an exception.

```
None
int age = 15;

if (age < 18)
{
    throw new Exception("Not eligible");
}
```

19.6 Custom Exception

A custom exception allows an application to define its own exception type.

```
None
class InvalidAgeException : Exception
{
    public InvalidAgeException(string message)
        : base(message)
    {
    }
}
```

Usage:

```
None
throw new InvalidAgeException("Invalid age");
```

19.7 Common Exception Types

Exception	Cause
NullReferenceException	Accessing a member through null
DivideByZeroException	Dividing an integer by zero
FormatException	Invalid conversion format
IndexOutOfRangeException	Invalid array index
FileNotFoundException	File not found
InvalidOperationException	Operation is invalid for current state

19.8 Best Practices

- Catch specific exceptions whenever possible.
- Do not silently ignore exceptions.
- Use finally or using for resource cleanup.
- Avoid using exceptions for ordinary control flow.
- Log useful error information.
- Do not expose sensitive internal error details to users.

---

20. COLLECTIONS AND GENERICS

20.1 What are Collections?

Collections are classes used to store and manage groups of objects.

Unlike arrays, many collections can dynamically grow or shrink.

20.2 Array vs Collection

Array	Collection
Usually fixed size after creation	Many collections grow dynamically
Stores elements of a declared element type	Depends on collection type
Simple and efficient	Provides more operations
Example: int[]	Example: List<int>

20.3 List<T>

List is a generic collection that stores elements of a specified type.

```
None
List<int> numbers = new List<int>();

numbers.Add(10);
numbers.Add(20);
numbers.Add(30);

foreach (int number in numbers)
{
    Console.WriteLine(number);
}
```

Output:

```
None
10
20
30
```

Important methods:

Method	Purpose
Add()	Adds an element
AddRange()	Adds multiple elements
Remove()	Removes a specified element
RemoveAt()	Removes by index
Contains()	Checks an element
Clear()	Removes all elements
Sort()	Sorts elements
Count	Gets number of elements

20.4 Dictionary<TKey,TValue>

Dictionary stores data as key-value pairs.

Each key must be unique.

```
None
Dictionary<int, string> students = new Dictionary<int, string>();

students.Add(1, "Divya");
students.Add(2, "John");

Console.WriteLine(students[1]);
```

Output:

```
None
Divya
```

20.5 HashSet<T>

HashSet stores unique elements and does not maintain a guaranteed insertion order.

```
None
HashSet<int> numbers = new HashSet<int>();

numbers.Add(10);
numbers.Add(20);
numbers.Add(10);

Console.WriteLine(numbers.Count);
```

Output:

```
None
2
```

20.6 Queue<T>

Queue follows FIFO (First In, First Out).

```
None
Queue<string> queue = new Queue<string>();

queue.Enqueue("A");
queue.Enqueue("B");

Console.WriteLine(queue.Dequeue());
```

Output:

```
None
A
```

20.7 Stack<T>

Stack follows LIFO (Last In, First Out).

```
None
Stack<int> stack = new Stack<int>();
```

```
stack.Push(10);
stack.Push(20);

Console.WriteLine(stack.Pop());
```

Output:

```
None
20
```

20.8 Generics

Generics allow classes, methods and interfaces to work with different data types while maintaining type safety.

Example:

```
None
class Box<T>
{
    public T Value { get; set; }

    public Box(T value)
    {
        Value = value;
    }
}
```

Usage:

```
None
Box<int> numberBox = new Box<int>(100);

Box<string> nameBox = new Box<string>("Divya");

Console.WriteLine(numberBox.Value);
Console.WriteLine(nameBox.Value);
```

Output:

```
None
100
Divya
```

20.9 Advantages of Generics

1. Type safety.
2. Code reusability.
3. Reduced explicit casting.
4. Better compile-time checking.
5. Reusable data structures and methods.

20.10 Collection Comparison

Collection	Behaviour	Use Case
List<T>	Ordered collection	Student list
Dictionary<K,V>	Key-value pairs	Employee lookup
HashSet<T>	Unique values	Removing duplicates
Queue<T>	FIFO	Task processing
Stack<T>	LIFO	Undo operations
Array	Fixed size	Fixed-size data

IMPORTANT C# KEYWORDS QUICK REFERENCE

Keyword	Purpose
class	Defines a class
struct	Defines a value type
interface	Defines a contract
namespace	Organizes types
public	Public accessibility

private	Private accessibility
protected	Derived-type accessibility
static	Class-level member
const	Compile-time constant
readonly	Read-only field
var	Implicit local variable typing
new	Creates an instance or hides an inherited member
this	Refers to the current instance
base	Refers to the base class
virtual	Allows overriding
override	Overrides an inherited virtual member
abstract	Declares an incomplete base type/member
sealed	Prevents inheritance or further overriding
interface	Declares an interface
ref	Passes an argument by reference
out	Passes an output argument by reference
try	Defines exception-prone code
catch	Handles exceptions
finally	Defines cleanup code
throw	Raises an exception
return	Returns control or a value
break	Exits a loop or switch
continue	Skips the current loop iteration
async	Declares an asynchronous method or lambda
await	Awaits an asynchronous operation

IMPORTANT DIFFERENCES FOR INTERVIEWS

Concept 1	Concept 2	Main Difference
C#	.NET	Language vs platform
Value type	Reference type	Value storage vs object reference
const	readonly	Compile-time constant vs runtime initialization allowed
Parse	TryParse	Exception on invalid input vs success indicator
while	do-while	Condition checked before vs after
Array	List	Fixed size vs dynamically resizable
String	StringBuilder	Immutable vs mutable
Class	Object	Blueprint vs instance
Constructor	Method	Object initialization vs operation
Overloading	Overriding	Different signatures vs inherited virtual member implementation
Abstract class	Interface	Shared base implementation vs contract
Encapsulation	Abstraction	Data access control vs implementation hiding
ref	out	Must be initialized before passing vs must be assigned by method
Stack	Queue	LIFO vs FIFO
Authentication	Authorization	Identity verification vs permission checking

---

IMPORTANT INTERVIEW QUESTIONS

1. What is C#?
2. What is the difference between C# and .NET?
3. What is CLR?
4. What is JIT compilation?
5. What is Garbage Collection?

6. What is the difference between value types and reference types?
7. What is the difference between var and dynamic?
8. What is the difference between const and readonly?
9. What is type casting?
10. What is the difference between Parse and TryParse?
11. What is the difference between while and do-while?
12. What is the difference between break and continue?
13. What is an array?
14. What is the difference between an array and List?
15. Why are strings immutable?
16. What is StringBuilder?
17. What is method overloading?
18. What is the difference between ref and out?
19. What is a class and an object?
20. What is a constructor?
21. What are the four pillars of OOP?
22. What is inheritance?
23. What is polymorphism?
24. What is method overriding?
25. What is the difference between virtual and override?
26. What is abstraction?
27. What is encapsulation?
28. What is an interface?
29. What is the difference between an interface and abstract class?
30. What are access modifiers?
31. What is exception handling?
32. What is the purpose of finally?
33. What is a custom exception?
34. What are generics?
35. What is the difference between Stack and Queue?

PRACTICAL PROGRAMS FOR REVISION

Practice implementing these programs without referring to the solutions.

1. Print Hello World.
2. Read and display student details.
3. Calculate the sum of two numbers.
4. Find whether a number is even or odd.
5. Find the largest of three numbers.
6. Calculate grade based on marks.
7. Display multiplication tables.
8. Find factorial of a number.
9. Check whether a number is prime.

10. Reverse a number.
11. Reverse a string.
12. Check whether a string is a palindrome.
13. Find the largest element in an array.
14. Sort an array.
15. Create a calculator using methods.
16. Implement method overloading.
17. Create Student and Employee classes.
18. Demonstrate constructor overloading.
19. Implement inheritance using Employee and Manager.
20. Demonstrate runtime polymorphism.
21. Implement encapsulation using BankAccount.
22. Create an abstract Shape class.
23. Implement multiple interfaces.
24. Handle invalid input using exceptions.
25. Manage student records using List and Dictionary.

FINAL QUICK REVISION CHECKLIST

Fundamentals

- C# and .NET
- CLR, JIT, BCL and GC
- Program structure
- Variables and constants
- Data types
- Type conversion

Programming Logic

- Operators
- Conditional statements
- Loops
- Arrays
- Strings
- Methods

Object-Oriented Programming

- Classes and objects
- Constructors
- Inheritance

- Polymorphism
- Abstraction
- Encapsulation
- Interfaces
- Access modifiers

Error Handling and Collections

- Exception handling
- Custom exceptions
- Arrays and collections
- Generics
- List
- Dictionary
- Stack and Queue

END OF ASSIGNMENT 3 – C# FUNDAMENTALS

Key Learning Outcome:

After completing this module, you should be able to write basic C# programs, work with data types and methods, implement object-oriented programming concepts, handle exceptions and use collections to manage data.

Next Assignment: Assignment 4 – Modern C# & .NET

Topics include delegates, Action, Func, Predicate, lambda expressions, events, extension methods, LINQ, asynchronous programming and modern .NET features.